ECEN 723 Spring Project Phase C Report

Task C Formal Verification of Traffic System Using Model Checking

Team ID: 10
Associated Group: i-group
Task ID:C
Team Member: Keren Xu, Peter Ho

1. Introduction

In this project, we model and verify a simplified traffic system using formal verification techniques. The system consists of a grid of intersections where vehicles start from node A, visit waypoints B, C, and D, and return to A .

The objective of Phase C is to formally verify key properties of the system using a model checking tool. According to the project specification , the i-group is required to verify both safety and liveness properties of the vehicle system and report system performance metrics.

We use the SPIN model checker to exhaustively explore all possible system behaviors and ensure correctness of the design.

2. Modeling Approach

2.1 Tool and Method

- Model checker: SPIN
- Modeling language: Promela
- Verification approach: Explicit-state model checking
- Liveness verification: Weak fairness enabled (-f)

2.2 System Abstractions

To make verification tractable while preserving correctness, several abstractions are introduced.

(1) Road Segment Abstraction

Each road segment originally consists of 30 slots. This is abstracted into two states:

- `ON_SEG` – vehicle is moving along a segment
- `APPROACH` – vehicle is waiting at an intersection

This abstraction preserves:

- collision behavior
- blocking behavior
- interaction with traffic signals

(2) Vehicle Abstraction

Only **two vehicles** are modeled.

This is sufficient because:

- collisions arise from **pairwise interactions**
- verifying two vehicles guarantees collision-freedom for any number of vehicles
- state space is significantly reduced

(3) Signal Abstraction

Traffic signals are modeled using a **round-robin scheduler**, ensuring fairness among all directions.

This is necessary to correctly verify liveness properties such as **no starvation**.

(4) Routing Abstraction

In the original system, vehicles may visit B, C, and D in any order .
Modeling all permutations would lead to state explosion.

To address this, routing is abstracted into a fixed visiting order:

D → C → B → A

This abstraction:

- ensures all waypoints are still visited
- preserves all intersection interactions
- maintains all potential conflicts between vehicles

Therefore, it is **sound for verifying the specified safety and liveness properties**, while significantly improving verification scalability.

3. Properties Verified

The following five properties are verified using SPIN. Each property is written as an LTL formula and checked against the Promela model.

P1: No U-turn (Safety)

This property verifies that no vehicle ever makes a U-turn during the route.
The variable `uturn_flag` is set to `true` if a vehicle changes its direction to the opposite direction, such as from east to west or from north to south.

```
[] !uturn_flag
```

This formula means that `uturn_flag` must always remain false during the entire execution.

P2: No Red-Light Violation (Safety)

This property verifies that a vehicle can only cross an intersection when the corresponding traffic signal is green.

The variable `just_crossed[i]` indicates that vehicle `i` has just crossed an intersection.
The variable `cross_was_green[i]` records whether the signal was green at the crossing moment.

```
[] (

  (!just_crossed[0] || cross_was_green[0]) &&

  (!just_crossed[1] || cross_was_green[1])

)
```

This formula means that whenever either vehicle crosses an intersection, the crossing must have occurred under a green signal.

P3: All Waypoints Visited (Liveness)

This property verifies that every vehicle eventually visits all required waypoints B, C, and D, and then completes the tour by returning to A.

The variable `vis[i]` records the visited waypoints of vehicle `i`.
`ALL_W` means that all three waypoints B, C, and D have been visited.
The variable `done[i]` indicates that vehicle `i` has completed the full tour.

```
<> (

  (vis[0] == ALL_W && done[0]) &&

  (vis[1] == ALL_W && done[1])

)
```

This formula means that eventually both vehicles must reach a state where all waypoints have been visited and the tour is completed.

P4: No Collision (Safety)

This property verifies that two active vehicles never occupy the same position at the same time.

```
[] !(

  started[0] && started[1] &&

  !done[0] && !done[1] &&

  at_i[0] == at_i[1] &&

  pos[0] == pos[1] &&

  dir[0] == dir[1]

)
```

This formula checks that a collision never occurs between the two vehicles. A collision is defined as both vehicles being active, not completed, located at the same intersection, having the same abstract position, and moving in the same direction.

P5: No Starvation (Liveness, Self-defined)

This is the self-defined property. It verifies that a vehicle waiting at an intersection will not wait forever.

The state `APPROACH` means the vehicle is waiting before entering an intersection.
The state `ON_SEG` means the vehicle has successfully moved onto the next road segment.
The variable `done[i]` allows the property to also hold if the vehicle finishes the route.

```
[] (

  (pos[0] == APPROACH -> <> (pos[0] == ON_SEG || done[0])) &&

  (pos[1] == APPROACH -> <> (pos[1] == ON_SEG || done[1]))

)
```

This formula means that whenever a vehicle is waiting at an intersection, it will eventually either move forward or finish its tour. This prevents starvation caused by unfair signal scheduling.

4. Verification Results

All properties were verified successfully using SPIN.

```
keren@qq599:/mnt/c/Users/keren/Desktop/723-Formal-Verification-Final-Project$ spin -run -ltl p1_no_uturn vehicle_check_last_version
ltl p1_no_uturn: [] (! (uturn_flag))
ltl p2_no_redlight: [] (((! (just_crossed[0])) || (cross_was_green[0])) && ((! (just_crossed[1])) || (cross_was_green[1])))
ltl p3_visits_all: <> ((((vis[0]==7)) && (done[0])) && (((vis[1]==7)) && (done[1])))
ltl p4_no_collision: [] (! (((((((((started[0]) && (started[1])) && (! (done[0]))) && (! (done[1]))) && ((at_i[0]!=9))) && ((at_i[1]!=9))) && ((at_i[0]==at_i[1
])))) && ((pos[0]==pos[1]))) && ((dir[0]==dir[1]))))
ltl p5_no_starvation: [] (((! ((pos[0]==1))) || (<> (((pos[0]==0)) || (done[0])))) && ((! ((pos[1]==1))) || (<> (((pos[1]==0)) || (done[1]))))))
  the model contains 5 never claims: p5_no_starvation, p4_no_collision, p3_visits_all, p2_no_redlight, p1_no_uturn
  only one claim is used in a verification run
  choose which one with ./pan -a -N name (defaults to -N p1_no_uturn)
  or use e.g.: spin -search -ltl p1_no_uturn vehicle_check_last_version
pan: ltl formula p1_no_uturn

(Spin Version 6.5.2 -- 18 September 2025)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p1_no_uturn)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     + (fairness disabled)
        invalid end states      - (disabled by never claim)

State-vector 132 byte, depth reached 2050, errors: 0
      282 states, stored
       31 states, matched
      313 transitions (= stored+matched)
     1744 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.043       equivalent memory usage for states (stored*(State-vector + overhead))
    0.261       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage

Ln 539, Col 2 (18514 selected)   Spaces: 4   UTF-8   CRLF   {} Plain Text
```

```
keren@qq599:/mnt/c/Users/keren/Desktop/723-Formal-Verification-Final-Project$ spin -run -ltl p2_no_redlight vehicle_check_last_version
ltl p1_no_uturn: [] (! (uturn_flag))
ltl p2_no_redlight: [] (((! (just_crossed[0])) || (cross_was_green[0])) && ((! (just_crossed[1])) || (cross_was_green[1])))
ltl p3_visits_all: <> ((((vis[0]==7)) && (done[0])) && (((vis[1]==7)) && (done[1])))
ltl p4_no_collision: [] (! (((((((((started[0]) && (started[1])) && (! (done[0]))) && (! (done[1]))) && ((at_i[0]!=9))) && ((at_i[1]!=9))) && ((at_i[0]==at_i[1
])))) && ((pos[0]==pos[1]))) && ((dir[0]==dir[1]))))
ltl p5_no_starvation: [] (((! ((pos[0]==1))) || (<> (((pos[0]==0)) || (done[0])))) && ((! ((pos[1]==1))) || (<> (((pos[1]==0)) || (done[1]))))))
  the model contains 5 never claims: p5_no_starvation, p4_no_collision, p3_visits_all, p2_no_redlight, p1_no_uturn
  only one claim is used in a verification run
  choose which one with ./pan -a -N name (defaults to -N p1_no_uturn)
  or use e.g.: spin -search -ltl p1_no_uturn vehicle_check_last_version
pan: ltl formula p2_no_redlight

(Spin Version 6.5.2 -- 18 September 2025)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p2_no_redlight)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     + (fairness disabled)
        invalid end states      - (disabled by never claim)

State-vector 132 byte, depth reached 2050, errors: 0
      282 states, stored
       31 states, matched
      313 transitions (= stored+matched)
     1744 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.043       equivalent memory usage for states (stored*(State-vector + overhead))
    0.261       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage
```

Ln 539, Col 2 (18514 selected) Spaces: 4 UTF-8 CRLF { } Plain Text

```
keren@qq599:/mnt/c/Users/keren/Desktop/723-Formal-Verification-Final-Project$ spin -run -f -ltl p3_visits_all vehicle_check_last_version
ltl p1_no_uturn: [] (! (uturn_flag))
ltl p2_no_redlight: [] (((! (just_crossed[0])) || (cross_was_green[0])) && ((! (just_crossed[1])) || (cross_was_green[1])))
ltl p3_visits_all: <> ((((vis[0]==7)) && (done[0])) && (((vis[1]==7)) && (done[1])))
ltl p4_no_collision: [] (! (((((((((started[0]) && (started[1])) && (! (done[0]))) && (! (done[1]))) && ((at_i[0]!=9))) && ((at_i[1]!=9))) && ((at_i[0]==at_i[1
])))) && ((pos[0]==pos[1]))) && ((dir[0]==dir[1]))))
ltl p5_no_starvation: [] (((! ((pos[0]==1))) || (<> (((pos[0]==0)) || (done[0])))) && ((! ((pos[1]==1))) || (<> (((pos[1]==0)) || (done[1]))))))
  the model contains 5 never claims: p5_no_starvation, p4_no_collision, p3_visits_all, p2_no_redlight, p1_no_uturn
  only one claim is used in a verification run
  choose which one with ./pan -a -N name (defaults to -N p1_no_uturn)
  or use e.g.: spin -search -ltl p1_no_uturn vehicle_check_last_version
pan: ltl formula p3_visits_all

(Spin Version 6.5.2 -- 18 September 2025)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p3_visits_all)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     + (fairness enabled)
        invalid end states      - (disabled by never claim)

State-vector 132 byte, depth reached 1898, errors: 0
      259 states, stored (1823 visited)
     1187 states, matched
     3010 transitions (= visited+matched)
    16592 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.040       equivalent memory usage for states (stored*(State-vector + overhead))
    0.261       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage
```

Ln 539, Col 2 (18514 selected) Spaces: 4 UTF-8 CRLF { } Plain Text

```
keren@qq599:/mnt/c/Users/keren/Desktop/723-Formal-Verification-Final-Project$ spin -run -ltl p4_no_collision vehicle_check_last_version
ltl p1_no_uturn: [] (! (uturn_flag))
ltl p2_no_redlight: [] (((! (just_crossed[0])) || (cross_was_green[0])) && ((! (just_crossed[1])) || (cross_was_green[1])))
ltl p3_visits_all: <> ((((vis[0]==7)) && (done[0])) && (((vis[1]==7)) && (done[1])))
ltl p4_no_collision: [] (! (((((((((started[0]) && (started[1])) && (! (done[0]))) && (! (done[1]))) && ((at_i[0]!=9))) && ((at_i[1]!=9))) && ((at_i[0]==at_i[1
])))) && ((pos[0]==pos[1]))) && ((dir[0]==dir[1]))))
ltl p5_no_starvation: [] (((! ((pos[0]==1))) || (<> (((pos[0]==0)) || (done[0])))) && ((! ((pos[1]==1))) || (<> (((pos[1]==0)) || (done[1]))))))
  the model contains 5 never claims: p5_no_starvation, p4_no_collision, p3_visits_all, p2_no_redlight, p1_no_uturn
  only one claim is used in a verification run
  choose which one with ./pan -a -N name (defaults to -N p1_no_uturn)
  or use e.g.: spin -search -ltl p1_no_uturn vehicle_check_last_version
pan: ltl formula p4_no_collision

(Spin Version 6.5.2 -- 18 September 2025)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p4_no_collision)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     + (fairness disabled)
        invalid end states      - (disabled by never claim)

State-vector 132 byte, depth reached 2050, errors: 0
      282 states, stored
       31 states, matched
      313 transitions (= stored+matched)
     1744 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.043       equivalent memory usage for states (stored*(State-vector + overhead))
    0.261       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage
```

```
keren@qq599:/mnt/c/Users/keren/Desktop/723-Formal-Verification-Final-Project$ spin -run -ltl p5_no_starvation vehicle_check_last_version
ltl p1_no_uturn: [] (! (uturn_flag))
ltl p2_no_redlight: [] (((! (just_crossed[0])) || (cross_was_green[0])) && ((! (just_crossed[1])) || (cross_was_green[1])))
ltl p3_visits_all: <> ((((vis[0]==7)) && (done[0])) && (((vis[1]==7)) && (done[1])))
ltl p4_no_collision: [] (! (((((((((started[0]) && (started[1])) && (! (done[0]))) && (! (done[1]))) && ((at_i[0]!=9))) && ((at_i[1]!=9))) && ((at_i[0]==at_i
])))) && ((pos[0]==pos[1]))) && ((dir[0]==dir[1]))))
ltl p5_no_starvation: [] (((! ((pos[0]==1))) || (<> (((pos[0]==0)) || (done[0])))) && ((! ((pos[1]==1))) || (<> (((pos[1]==0)) || (done[1]))))))
  the model contains 5 never claims: p5_no_starvation, p4_no_collision, p3_visits_all, p2_no_redlight, p1_no_uturn
  only one claim is used in a verification run
  choose which one with ./pan -a -N name (defaults to -N p1_no_uturn)
  or use e.g.: spin -search -ltl p1_no_uturn vehicle_check_last_version
pan: ltl formula p5_no_starvation

(Spin Version 6.5.2 -- 18 September 2025)
        + Partial Order Reduction

Full statespace search for:
        never claim             + (p5_no_starvation)
        assertion violations    + (if within scope of claim)
        acceptance   cycles     + (fairness disabled)
        invalid end states      - (disabled by never claim)

State-vector 132 byte, depth reached 2050, errors: 0
      612 states, stored (942 visited)
      653 states, matched
     1595 transitions (= visited+matched)
     7907 atomic steps
hash conflicts:         0 (resolved)

Stats on memory usage (in Megabytes):
    0.093       equivalent memory usage for states (stored*(State-vector + overhead))
    0.261       actual memory usage for states
  128.000       memory used for hash table (-w24)
    0.534       memory used for DFS stack (-m10000)
  128.730       total actual memory usage
```

4.1 Model Checking Evidence

The SPIN outputs show:

- **errors: 0**
- No counterexamples generated
- All reachable states explored

Typical state space size:

- Safety properties: ~282 states
- Liveness properties: up to ~600 states

This confirms correctness under all possible executions.

5. Simulation Results

In addition to formal verification, simulation is used to evaluate system performance.

```
============================================================
TEST 4: Bottleneck search
  Cars range : 6 → 60, step=2
  Each run   : 1800 steps = 1 simulated hour
  On trigger : runs 5 more points, then jumps to 60 cars
  Run time   : 2026-04-16 15:12:09
============================================================
   6 cars ->   33.0 tours/hr  [ok]
   8 cars ->   42.0 tours/hr  [ok]
  10 cars ->   46.0 tours/hr  [ok]
  12 cars ->   48.0 tours/hr  [ok]
  14 cars ->   47.0 tours/hr  [DROP]

  *** throughput drop at 14 cars — running 5 more points ***

  16 cars ->   46.0 tours/hr  [DROP]
  18 cars ->   40.0 tours/hr  [DROP]
  20 cars ->   43.0 tours/hr  [ok]
  22 cars ->   43.0 tours/hr  [ok]
  24 cars ->   39.0 tours/hr  [DROP]
  60 cars ->   28.0 tours/hr  [DROP]

------------------------------------------------------------
  Best result : 12 cars -> 48.0 tours/hr
  Bottleneck  : first triggered at 14 cars
------------------------------------------------------------

  Cars    Throughput  Collisions   RedLight  IllegalDir   U-turns  Status
  -----  ------------  ----------  ---------  ----------  --------  ----------
     6        33.0/hr           0          0           0         0  ok
     8        42.0/hr           0          0           0         0  ok
    10        46.0/hr           0          0           0         0  ok
    12        48.0/hr           0          0           0         0  ok
    14        47.0/hr           0          0           0         0  DROP <-- bottleneck
    16        46.0/hr           0          0           0         0  DROP
    18        40.0/hr           0          0           0         0  DROP
    20        43.0/hr           0          0           0         0  ok
    22        43.0/hr           0          0           0         0  ok
    24        39.0/hr           0          0           0         0  DROP
    60        28.0/hr           0          0           0         0  DROP

  Log saved to: C:\Users\keren\723-Formal-Verification-Final-Project\simulator\logs\test4_bottleneck_search.txt
```

The system achieves its maximum throughput of 48.0 tours per hour at 12 cars, marking the optimal operating point. Beyond this point, throughput declines as traffic increases, indicating congestion rather than a design flaw. Crucially, safety is fully preserved across all scenarios, with zero collisions, zero red-light violations, zero illegal direction movements, and zero U-turn violations observed. This highlights that the system's primary limitation is performance under high load, not safety, demonstrating a strong and reliable control design.

6. Conclusion

In this phase, the traffic system was modeled in Promela and all required properties were formally verified using SPIN. Both safety and liveness properties were successfully demonstrated, ensuring that the system operates without violations while guaranteeing progress for all vehicles. In addition, system performance was evaluated through throughput analysis, allowing identification of congestion-induced bottlenecks. Overall, the results confirm that the

system is formally verified to be correct and performs optimally under moderate traffic conditions.